# Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas


# Automatización de recorridos móviles con Maestro

Proyecto final: TaskFlow Mobile – Gestor de Tareas


**Estudiante:** Javier Alejandro Guerra Zepeda


**Fecha:** 2026-10-31

# 1. Descripción de la aplicación

**TaskFlow Mobile** es una aplicación Android nativa, desarrollada con React Native 0.88, Expo SDK 58 y TypeScript, que permite gestionar tareas personales. Se distribuye como un APK *release* que incluye el código JavaScript empaquetado (motor Hermes), por lo que no depende de Expo Go ni de un servidor Metro en ejecución.

Funciones disponibles:

- Pantalla principal con el listado de tareas y un contador de pendientes y completadas.
- Formulario de nueva tarea con título obligatorio (máx. 80 caracteres) y descripción opcional.
- Validación: si el título está vacío o solo contiene espacios se muestra «El título es obligatorio» y no se guarda nada.
- Confirmación visible «Tarea guardada correctamente» tras un guardado exitoso.
- Marcar o desmarcar una tarea como completada (estado «Pendiente» / «Completada» y texto tachado).
- Persistencia local con AsyncStorage: las tareas se conservan al cerrar y reabrir la aplicación.
- Navegación entre pantallas con el botón «Volver», «Cancelar» y el botón atrás de Android; el formulario se adapta al teclado.

La aplicación no usa backend ni servicios externos: todos los datos se guardan en el almacenamiento privado de la app en el dispositivo. Esto la hace adecuada para probar recorridos móviles, porque incluye entrada de texto con el teclado de Android, validaciones, navegación, cambios de estado y un comportamiento propio del móvil: la recuperación de datos tras cerrar el proceso. Todos los controles tienen *testID* estables (por ejemplo `btn-nueva-tarea`, `input-titulo`, `btn-guardar`, `error-titulo`), que Maestro localiza sin usar coordenadas.

## 2. Entorno utilizado

| Elemento | Valor |
|---|---|
| Sistema operativo | Microsoft Windows 11 Home Single Language 10.0.26200 (build 26200) 64 bits |
| Equipo | AMD Ryzen 5 7520U with Radeon Graphics, 7.2 GB de RAM |
| Plataforma | Android (emulador oficial de Android SDK, aceleración WHPX) |
| Versión de Android | Android 15 (API 35) |
| Emulador | AVD «TaskFlow_API35», perfil Pixel 6, imagen google_apis x86_64, 1080×2400, 420 dpi, 2 GB RAM — serial ADB emulator-5554, modelo sdk_gphone64_x86_64 |
| Maestro CLI | 2.12.0 |
| Node.js / npm | Node.js v22.23.4 (compilación) y v22.10.0 (sistema) / npm 10.9.0 |
| Java / Android | java version "21.0.1" 2023-10-17 LTS; Android Debug Bridge version 1.0.41 / Version 37.0.1-15733141; Android Emulator 37.2.13.0 (build_id 16740729) (CL:N/A) |
| Framework | Expo SDK 58 (CLI 58.1.18), React Native 0.88.1, TypeScript 6, Gradle 9.4.1 |
| Versión de la app | 1.0.0 (versionCode 1) |
| Identificador de paquete | gt.edu.umg.taskflow |
| Almacenamiento | Local con @react-native-async-storage/async-storage 2.2.0 (sin backend) |
| Fecha de ejecución | 2026-10-31 |

## 3. Escenarios automatizados

| Escenario | Objetivo | Precondiciones | Acciones principales | Resultado esperado | Resultado observado |
|---|---|---|---|---|---|
| 01 – Registro exitoso `01-registro-exitoso.yaml` | Crear una tarea y verla en el listado. | App instalada; datos borrados con clearState; sin sesiones previas. | Abrir app, verificar pantalla principal, abrir formulario, escribir título y descripción, guardar. | Confirmación visible y tarea listada con el título correcto, estado Pendiente. | **APROBADO** (código de salida 0). Se mostró «Tarea guardada correctamente» y la tarea «Preparar informe de pruebas» apareció como Pendiente; contador «1 pendiente · 0 completadas». |
| 02 – Validación `02-validacion-entrada.yaml` | Rechazar una tarea sin título. | App instalada; listado vacío (clearState); formulario disponible. | Abrir formulario, dejar título vacío, escribir descripción, guardar; repetir con título de solo espacios; cancelar. | Mensaje de error, sin confirmación y sin registro inválido. | **APROBADO** (código de salida 0). Apareció «El título es obligatorio» en ambos intentos, no hubo confirmación y el listado siguió vacío («0 pendientes · 0 completadas»). |
| 03 – Persistencia `03-persistencia-reapertura.yaml` | Conservar datos tras cerrar y reabrir. | App instalada; limpieza solo al inicio; almacenamiento local operativo. | Crear tarea, stopApp, launchApp sin clearState, verificar, marcar completada, reabrir otra vez. | La tarea sigue tras reabrir y su estado Completada se refleja y se conserva. | **APROBADO** (código de salida 0). «Tarea persistente reapertura» siguió visible tras reabrir; al marcarla cambió a Completada («0 pendientes · 1 completada») y lo conservó tras una segunda reapertura. |

## 4. Ejecución y resultados

**Instalación.** El APK se compiló localmente con `scripts\compilar-apk.ps1` (`expo prebuild` + `gradlew assembleRelease`) y se instaló en el emulador con `adb install -r`. Se verificó con `dumpsys package` que el paquete instalado es `gt.edu.umg.taskflow`, versionName 1.0.0 y versionCode 1, el mismo `appId` de los flujos.

**Datos.** Cada escenario comienza con `helpers/iniciar-limpio.yaml`, que abre la app con `clearState: true` y comprueba que el listado está vacío. Los datos de prueba (títulos y descripciones) están definidos en la sección `env` de cada YAML.

**Ejecución.** Cada escenario se ejecutó por separado y después los tres juntos con `scripts\ejecutar-pruebas.ps1`, que llama a Maestro con reporte JUnit y guarda la salida de la terminal, el código de salida y las capturas:

```
maestro test maestro/01-registro-exitoso.yaml --format JUNIT --output
resultados/reportes/escenario-01-junit.xml --test-output-dir resultados/ejecuciones/... --udid emulator-5554
maestro test maestro --format JUNIT --output resultados/reportes/ejecucion-conjunta-junit.xml --udid
emulator-5554
```

**Verificaciones.** Además de los toques, cada flujo usa `assertVisible` y `assertNotVisible` sobre identificadores y textos (pantalla visible, formulario abierto, contenido de los campos, mensaje de confirmación o de error, título y estado de la tarea, contador), y `extendedWaitUntil` como espera condicionada en lugar de pausas fijas.

**Resultados devueltos por Maestro** (tomados de los reportes JUnit y de los registros):

| Ejecución | Inicio | Flujos | Duración | Código salida | Resultado |
|---|---|---|---|---|---|
| Escenario 01 | 2026-10-31 19:36:44 | 1/1 aprobados | 50.5 s | 0 | APROBADO |
| Escenario 02 | 2026-10-31 19:38:27 | 1/1 aprobados | 52.3 s | 0 | APROBADO |
| Escenario 03 | 2026-10-31 19:40:05 | 1/1 aprobados | 60.5 s | 0 | APROBADO |
| Conjunta (3 flujos) | 2026-10-31 19:41:51 | 3/3 aprobados | 152.3 s | 0 | APROBADO |

Además de las ejecuciones de la tabla, hubo una primera ejecución del escenario 01 (`resultados/ejecuciones/escenario-01-20261031-193356`), también aprobada con código 0; se repitió solo para corregir la codificación del registro y la captura de la terminal, y se conserva en el repositorio. En total, los siete archivos `commands.json` de Maestro (uno por flujo ejecutado) registran todos sus comandos con estado COMPLETED (`resultados/resumen-comandos-maestro.txt`).

En la ejecución conjunta Maestro corrió los tres flujos en orden (definido en `maestro/config.yaml`) sobre la misma instalación. Como cada flujo limpia su propio estado al comenzar, ninguno depende de los datos que deja el anterior.

## 5. Evidencias gráficas

Todas las capturas del dispositivo se tomaron con el comando `takeScreenshot` de Maestro durante las ejecuciones registradas. Las capturas de terminal son imágenes de la ventana real de PowerShell en la que corrió cada ejecución.

## 5.1 Escenario 01 – Registro exitoso

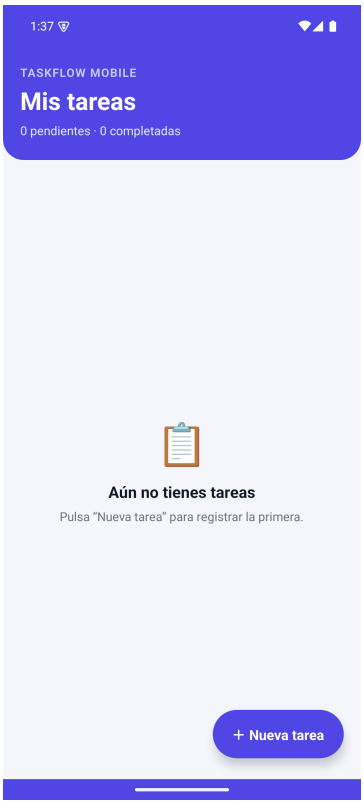



*Figura 1. Pantalla principal de TaskFlow Mobile (estado limpio).*

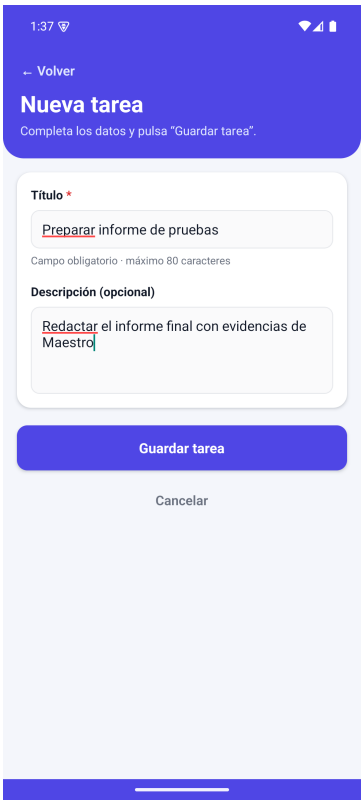



*Figura 2. Formulario completado.*

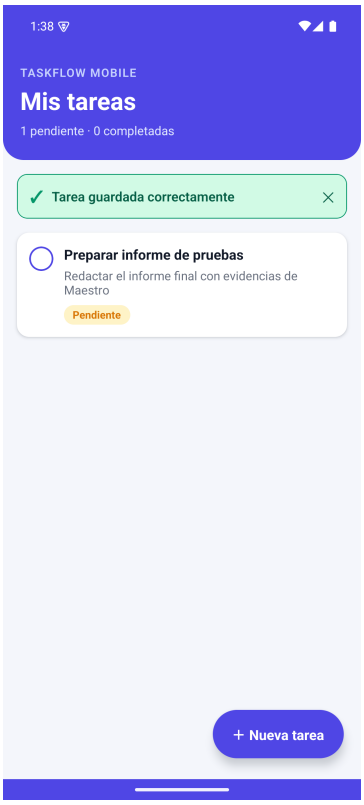



*Figura 3. Confirmación y tarea registrada.*

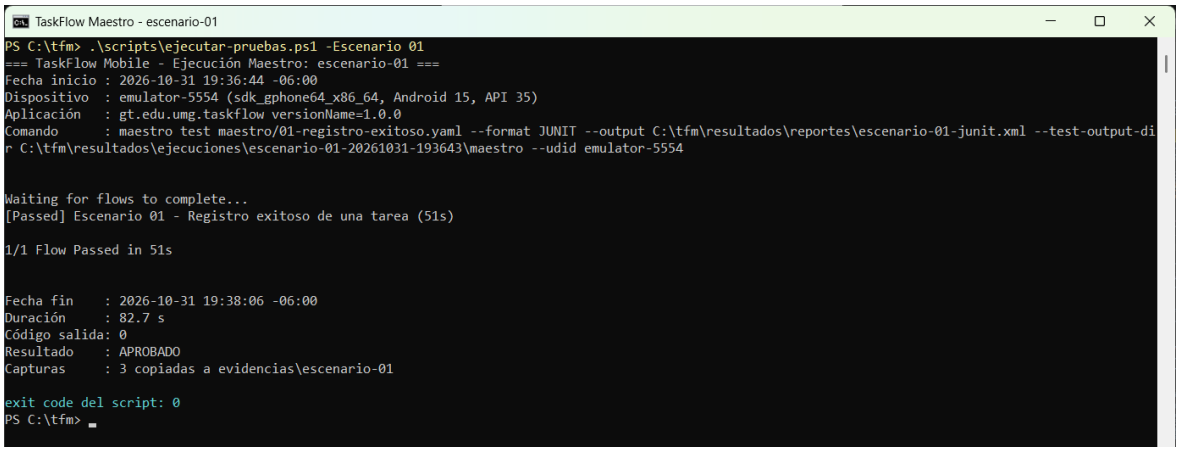



*Figura 4. Resultado de Maestro para el escenario 01: ventana de PowerShell de la ejecución (área con texto; original completo en evidencias/escenario-01/).*

## 5.2 Escenario 02 – Validación de campo obligatorio




*Figura 5. Formulario con el título vacío.*

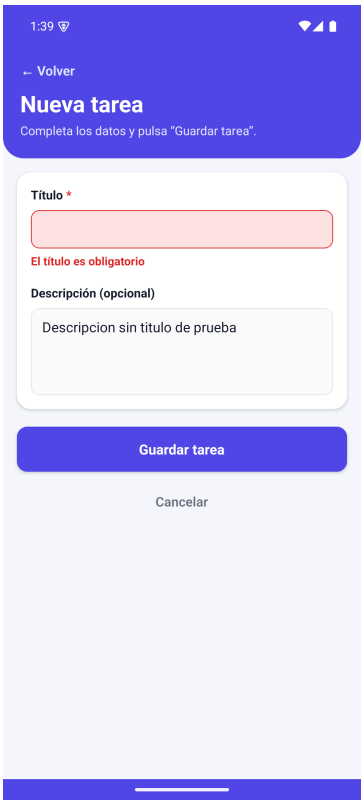



*Figura 6. Mensaje de validación.*

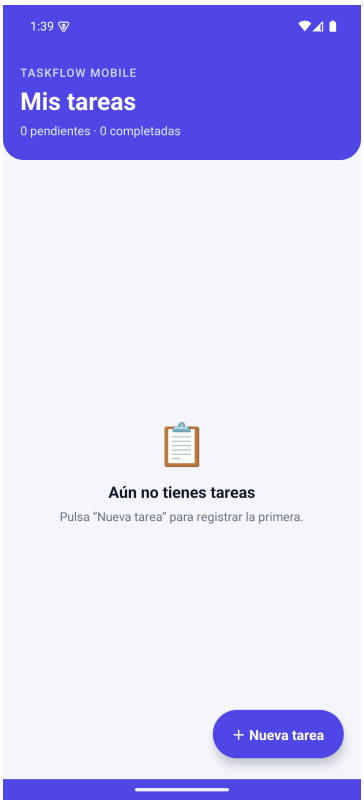



*Figura 7. Listado sin registros tras el rechazo.*

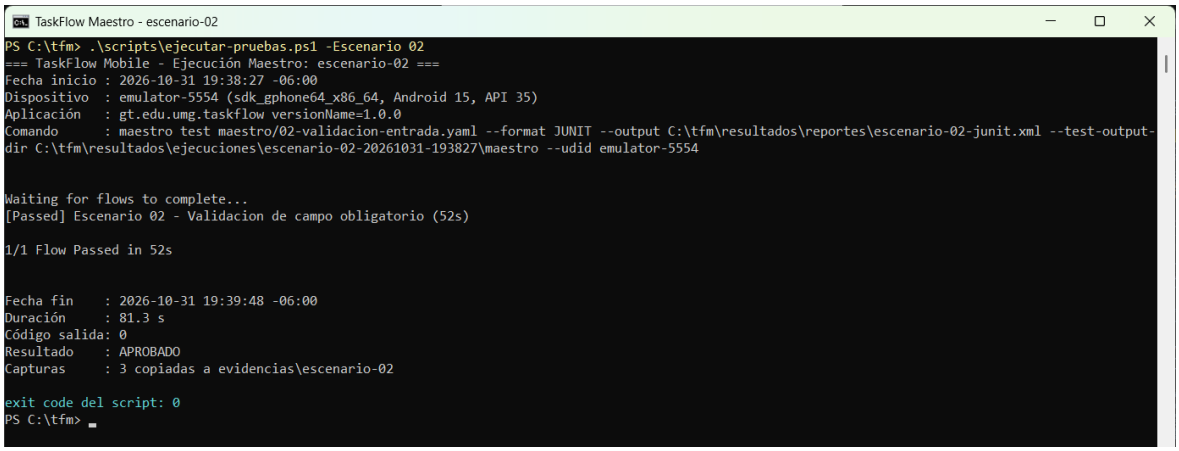



*Figura 8. Resultado de Maestro para el escenario 02: ventana de PowerShell de la ejecución (área con texto; original completo en evidencias/escenario-02/).*

## 5.3 Escenario 03 – Reapertura y recuperación del estado

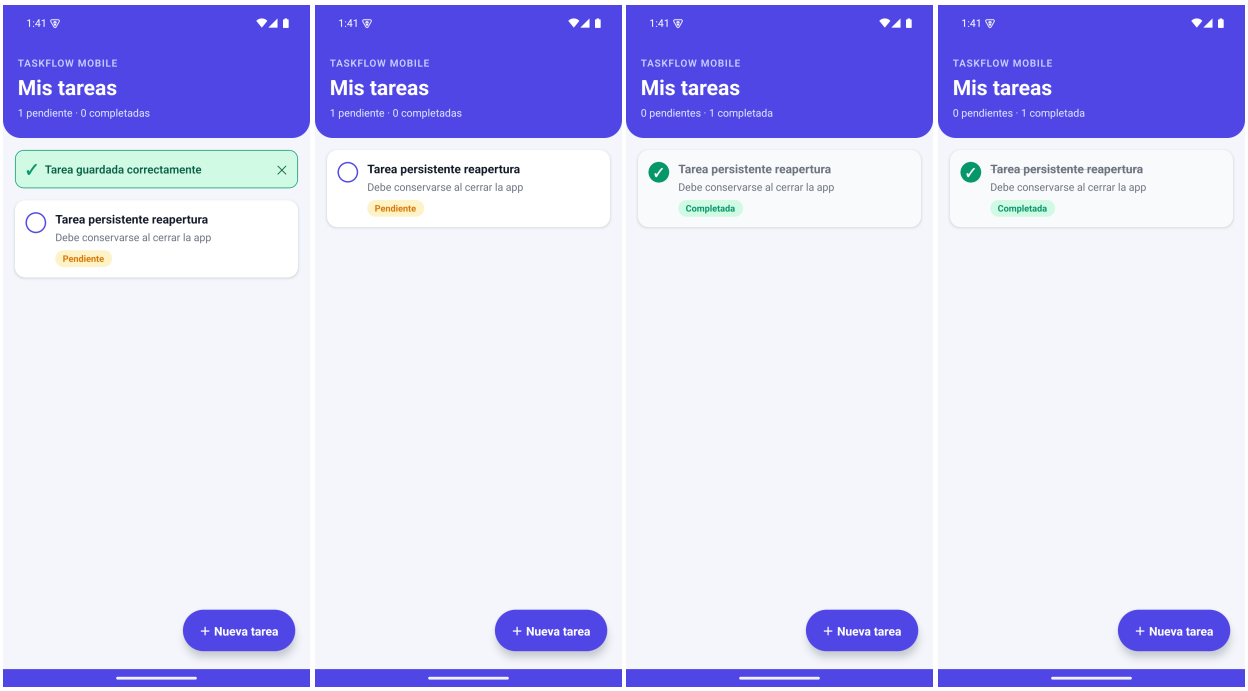



*Figura 9. Tarea creada antes del cierre.*




*Figura 10. App reabierta: tarea conservada.*




*Figura 11. Tarea marcada como completada.*




*Figura 12. Estado conservado tras otra reapertura.*

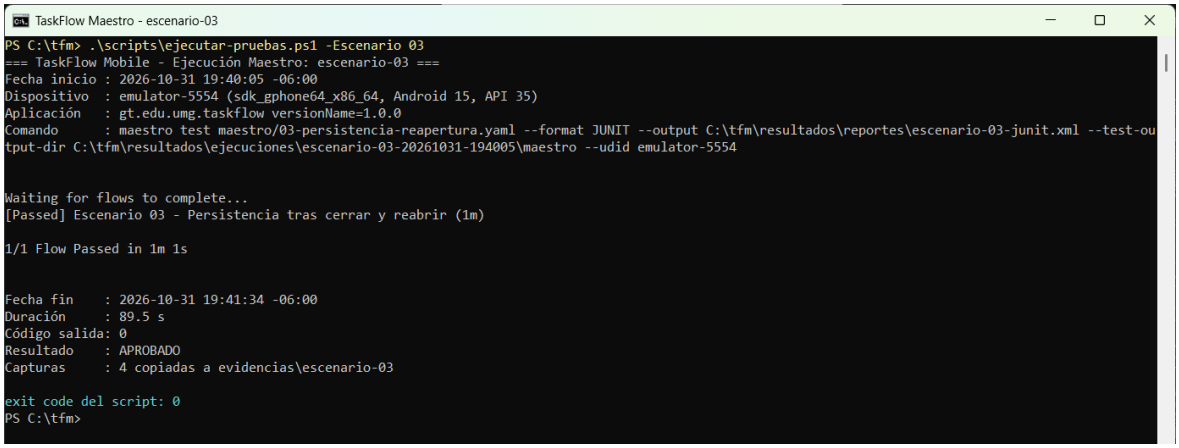



*Figura 13. Resultado de Maestro para el escenario 03: ventana de PowerShell de la ejecución (área con texto; original completo en evidencias/escenario-03/).*

### 5.4 Ejecución conjunta

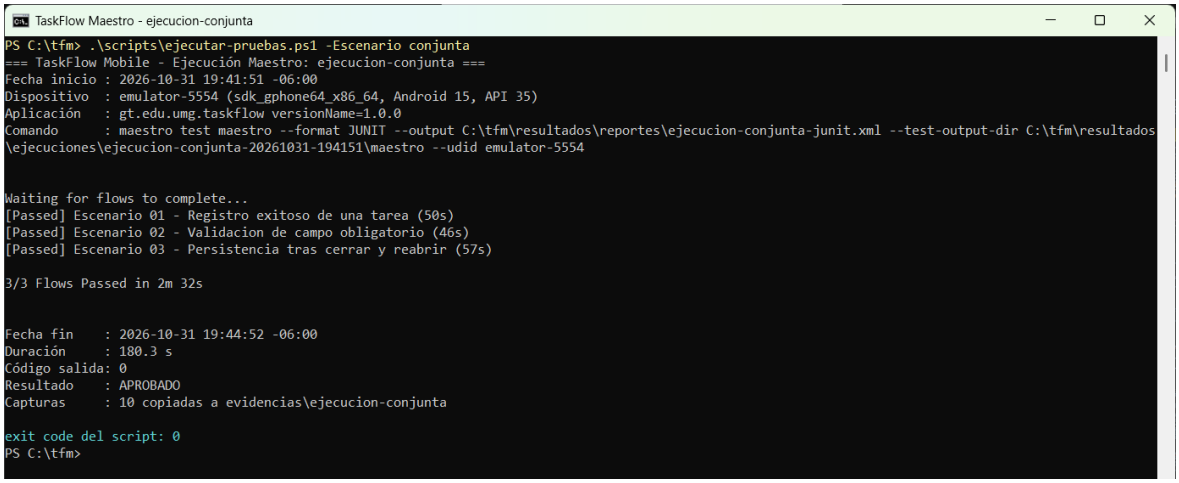



*Figura 14. Resultado de Maestro para la ejecución conjunta de los tres escenarios: ventana de PowerShell (área con texto; original completo en evidencias/ejecucion-conjunta/).*

## 6. Preparación y limpieza

- **Independencia:** los tres escenarios empiezan con `runFlow: helpers/iniciar-limpio.yaml`. Ese helper usa `launchApp` con `clearState: true` y comprueba que se ve «Aún no tienes tareas» y «0 pendientes · 0 completadas». Así cada escenario se puede ejecutar solo, en cualquier orden y varias veces.
- **Alcance de la limpieza:** `clearState` borra únicamente los datos privados del paquete `gt.edu.umg.taskflow` en el emulador de pruebas (igual que «Borrar datos» en Ajustes). La aplicación no tiene ninguna función oculta de restablecimiento que pudiera afectar a usuarios reales.
- **Persistencia (escenario 03):** la limpieza se hace solo una vez, antes de crear la tarea. Las reaperturas usan `stopApp` y `launchApp` con `clearState: false`, de modo que el proceso se cierra pero los datos de AsyncStorage se conservan.
- **Datos de prueba:** títulos y descripciones ficticios definidos en el bloque `env` de cada flujo; no se usan datos personales.
- **Limpieza manual opcional:** `scripts\limpiar-datos.ps1` ejecuta `adb shell pm clear gt.edu.umg.taskflow`.

## 7. Dependencias y limitaciones

La aplicación usa **almacenamiento local** (AsyncStorage). No hay backend real ni simulado y no requiere conexión a Internet para funcionar. Limitaciones identificadas:

- Las pruebas se ejecutaron solo en el emulador descrito (Android 15, API 35, x86_64). No se probó en un dispositivo físico ni en otras versiones de Android, ni en iOS.
- El APK incluye las arquitecturas arm64-v8a y x86_64 (no armeabi-v7a ni x86), para reducir tiempo de compilación.
- El APK release está firmado con la clave de depuración que genera Expo; sirve para instalarlo directamente, pero no para publicarlo en Google Play, que exige una clave de subida propia.

- Los datos que Maestro teclea con `inputText` se escribieron sin tildes por precaución, ya que la escritura de caracteres no ASCII en Android puede no ser fiable; no se probó la entrada con acentos. Los textos de la interfaz y las aserciones sí usan tildes (por ejemplo «El título es obligatorio»).
- La carpeta `app/android` no se versiona: se genera con `expo prebuild` a partir de `app.json`.

## 8. Enlaces de entrega

| Recurso | Enlace |
|---|---|
| Repositorio GitHub | https://github.com/javier2002guerra-lab/Automatizaci-n-de-recorridos-m-viles-con-Maestro |
| Commit documentado | https://github.com/javier2002guerra-lab/Automatizaci-n-de-recorridos-m-viles-con-Maestro/commit/5ee32f03b67ccbf163f5c378d44f20f6cfbdd02f |
| APK instalable (Release v1.0.0) | https://github.com/javier2002guerra-lab/Automatizaci-n-de-recorridos-m-viles-con-Maestro/releases/download/v1.0.0/TaskFlowMobile-1.0.0.apk |
| Flujos YAML | https://github.com/javier2002guerra-lab/Automatizaci-n-de-recorridos-m-viles-con-Maestro/tree/main/maestro |
| Evidencias (capturas) | https://github.com/javier2002guerra-lab/Automatizaci-n-de-recorridos-m-viles-con-Maestro/tree/main/evidencias |
| Registros y reportes JUnit | https://github.com/javier2002guerra-lab/Automatizaci-n-de-recorridos-m-viles-con-Maestro/tree/main/resultados |

## 9. Resultado final

En las ejecuciones individuales se aprobaron **3 de 3** escenarios (01: APROBADO, 02: APROBADO, 03: APROBADO). En la ejecución conjunta Maestro informó **3 de 3** flujos aprobados, con resultado **APROBADO** (código de salida 0).

Con estos resultados se comprueba, en el emulador indicado, que TaskFlow Mobile permite registrar tareas, rechaza entradas sin título sin crear registros inválidos y conserva los datos y su estado después de cerrar y reabrir la aplicación. Los registros originales, los reportes JUnit y todas las capturas están publicados en el repositorio (commit 5ee32f0).